

Laporan Praktikum Kontrol Cerdas

Nama : Fryscha Febryanti Puspitasari
NIM : 224308008
Kelas : TKA 6A
Akun Github (Tautan) : <https://github.com/Fryscha>
Student Lab Assistant : Yulia Brilianty

1. Judul Percobaan

Real-time Track Geometry Detection using Canny & Stereo Vision

2. Tujuan Percobaan

Percobaan ini dilakukan dengan tujuan sebagai berikut.

- Menerapkan Canny Edge Detection untuk mendeteksi batas rel.
- Menggunakan dua kamera untuk akuisisi data secara real-time.
- Menggunakan Stereo Vision untuk memperoleh kedalaman geometri rel secara real-time.
- Menghitung parameter lebar rel, panjang rel, kemiringan rel, dan deformasi ballast.

3. Landasan Teori

Canny Edge Detection adalah algoritma deteksi tepi yang dirancang untuk mengidentifikasi tepi dalam citra dengan akurasi tinggi. Algoritma ini dikembangkan oleh John F. Canny pada tahun 1986 dan telah menjadi standar dalam pemrosesan citra digital. Canny Edge Detection bekerja melalui beberapa tahapan, termasuk penghalusan citra menggunakan filter Gaussian untuk mengurangi noise, perhitungan gradien intensitas untuk menemukan perubahan intensitas yang signifikan, dan penerapan non-maximum suppression untuk memastikan bahwa hanya tepi lokal maksimum yang dipertahankan. Proses ini diakhiri dengan double thresholding dan edge tracking by hysteresis untuk menghubungkan tepi yang terdeteksi. Algoritma ini terkenal karena kemampuannya dalam mendeteksi tepi yang halus dan akurat, menjadikannya pilihan yang populer dalam berbagai aplikasi pemrosesan citra.

Penggunaan dua kamera (stereo camera setup) memungkinkan pengambilan gambar simultan dari sudut pandang berbeda dalam waktu nyata (real-time). Hal ini penting agar data visual tentang kondisi geometri rel dapat diperoleh secara langsung tanpa penundaan signifikan.

Stereo vision memanfaatkan perbedaan posisi objek pada dua citra hasil tangkapan kamera kiri dan kanan (disparity) untuk menghitung kedalaman atau jarak objek terhadap kamera. Stereo vision memungkinkan pemetaan tiga dimensi geometri jalur rel secara real-time, termasuk variasi elevasi atau deformasi permukaan ballast di sekitar rel.

GitHub digunakan sebagai platform untuk version control dan dokumentasi proyek. Version control memungkinkan pengembang untuk melacak perubahan dalam kode dan berkolaborasi dengan orang lain secara efektif. Dokumentasi proyek mencakup deskripsi algoritma, implementasi kode, dan hasil eksperimen. Penggunaan GitHub memfasilitasi transparansi dan reproduktibilitas penelitian, serta memungkinkan orang lain untuk membangun di atas pekerjaan yang telah dilakukan. GitHub adalah *toolkit* untuk mengembangkan dan membandingkan algoritma

Python adalah salah satu bahasa pemrograman yang digunakan dalam pengembangan sistem kontrol cerdas. Bahasa pemrograman ini sangat diminati oleh pengguna karena kemudahan penggunaannya dan banyaknya pustaka yang tersedia.

4. Analisis dan Diskusi

Analisis Hasil:

Pembuatan setup stereo camera diawali dengan dua kamera USB diinisialisasi untuk menangkap gambar secara simultan dari sudut pandang kiri dan kanan secara real-time. Program ini memastikan bahwa kedua kamera dapat berfungsi dengan baik dan menampilkan video streaming secara paralel, yang merupakan dasar penting untuk akuisisi data stereo. Kemudian, pembuatan canny edge detection dilakukan dengan menerapkan algoritma Canny Edge Detection pada frame video yang diperoleh dari masing-masing kamera stereo. Proses ini meliputi konversi citra ke grayscale, penghalusan dengan Gaussian blur, lalu deteksi tepi menggunakan ambang batas tertentu. Hasilnya adalah peta tepi yang menonjolkan batas-batas objek seperti rel kereta api dalam gambar secara real-time. Lalu, dilakukan perhitungan peta kedalaman berdasarkan metode stereo vision menggunakan algoritma block matching

StereoBM dari OpenCV. Disparitas antara citra kiri dan kanan dihitung untuk menghasilkan depth map yang merepresentasikan jarak relatif objek terhadap kamera.

Diskusi:

Output dari kamera stereo yaitu menampilkan dua kamera. Kemudian, output canny edge detection berupa citra biner dengan garis-garis tepi jelas membantu mengidentifikasi batas fisik rel pada setiap frame secara langsung sehingga memudahkan ekstraksi fitur geometris jalur kereta. Lalu, output peta kedalaman menunjukkan variasi jarak permukaan rel terhadap posisi kamera dalam bentuk bounding box. Area dengan disparitas tinggi menandakan objek lebih dekat sedangkan nilai rendah menunjukkan objek lebih jauh.

5. Assignment

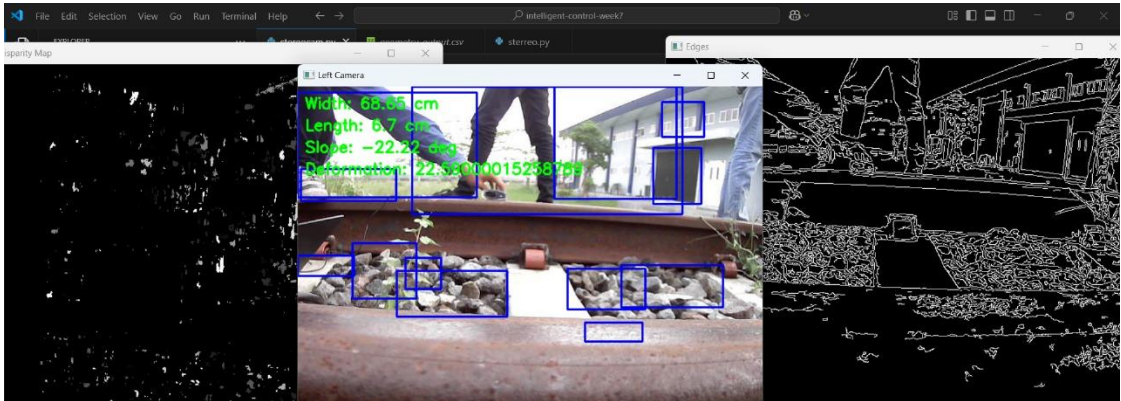
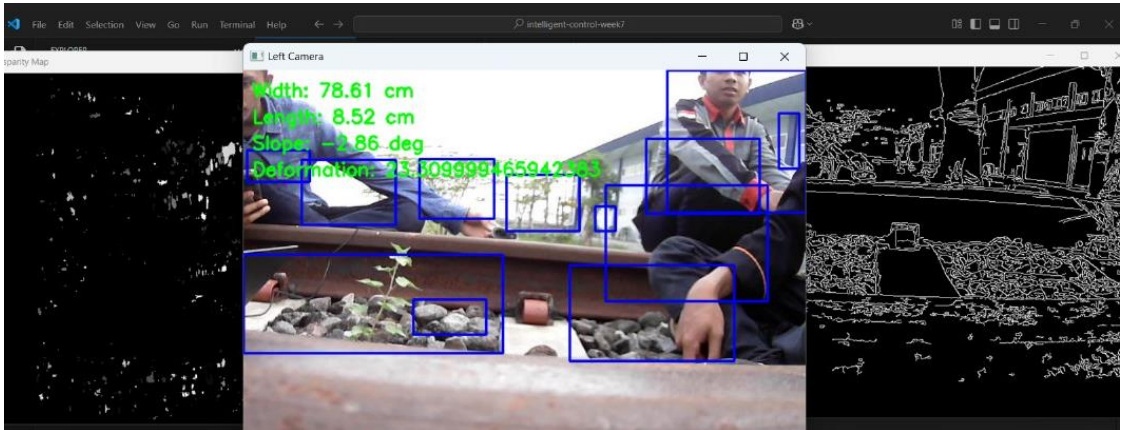
Topik percobaan ini adalah menambahkan fitur pengukuran akurasi dengan membandingkan hasil inspeksi manual dan inspeksi robot. Selain itu, memodifikasi kode agar dapat menyimpan data ke dalam format csv.

Hasil dari percobaan praktikum ini yaitu penambahan fitur pengukuran akurasi dengan membandingkan hasil inspeksi manual dan inspeksi robot memberikan kontribusi penting dalam mengevaluasi keandalan sistem deteksi geometri rel berbasis Canny Edge Detection dan Stereo Vision. Fitur ini memungkinkan validasi kuantitatif terhadap performa algoritma otomatis, sehingga dapat mengidentifikasi tingkat kesesuaian dan potensi kesalahan dalam pendeteksian parameter seperti lebar, kemiringan, dan deformasi rel. Selain itu, modifikasi kode untuk menyimpan data hasil inspeksi secara otomatis ke file CSV meningkatkan efisiensi dokumentasi serta memudahkan analisis data historis secara terstruktur. Integrasi kedua fitur tersebut memperkuat aplikasi praktis sistem kontrol cerdas ini dengan menyediakan alat evaluasi yang komprehensif sekaligus mendukung pelaporan yang lebih sistematis untuk pemeliharaan jalur kereta api yang lebih efektif dan terpercaya.

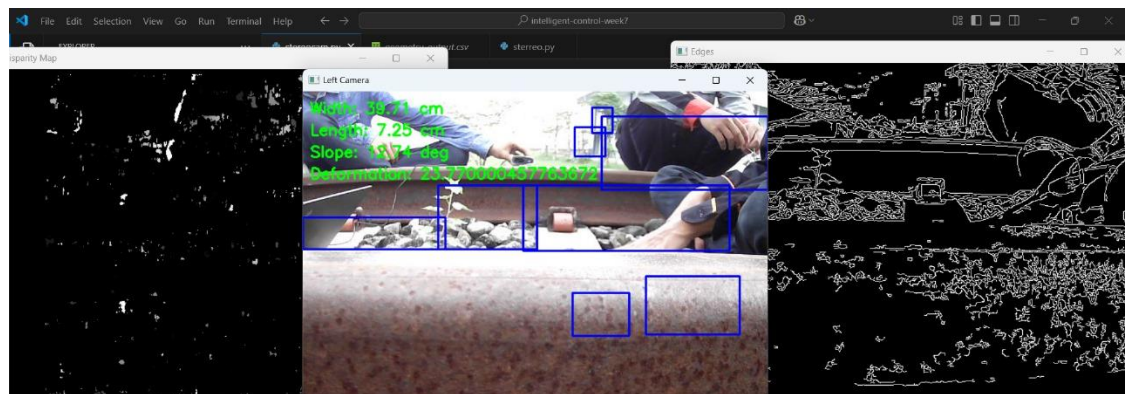
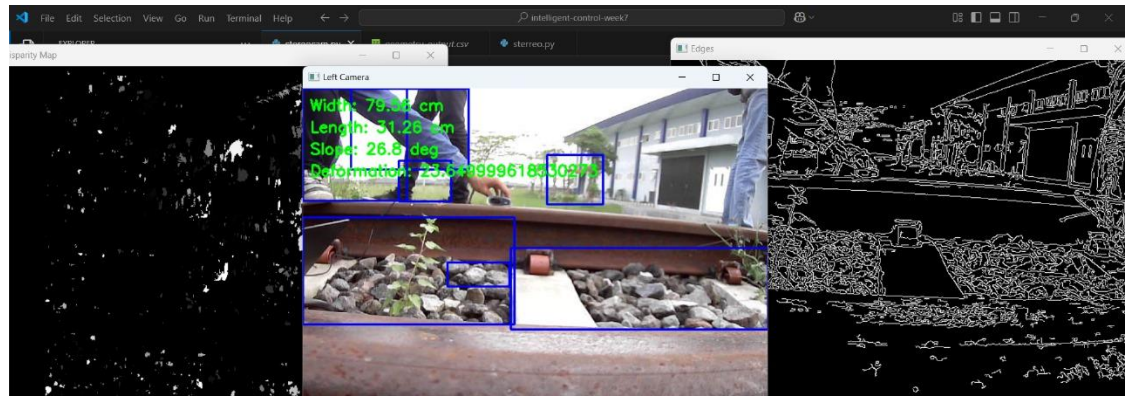
6. Data dan Output Hasil Pengamatan

Sajikan data dan hasil yang diperoleh selama percobaan. Gunakan tabel untuk menyajikan data jika diperlukan.

No Track	Lebar Rel (cm)	Panjang Rel (cm)	Kemiringan Rel (°)	Identifikasi Deformasi Ballast (cm)
1	68,65	6,7	-22,22	22,59
2	78,61	8,52	-2,86	23,3
3	79,56	31,26	26,8	23,64
4	39,71	7,25	12,74	23,77

No	Variabel	Hasil Pengamatan
1	Track 1	Output yang ditampilkan dapat berjalan dengan baik. 
2	Track 2	Output yang ditampilkan di bawah ini. 

3	Track 3	Output yang ditampilkan dapat berjalan dengan baik.
4	Track 4	Output yang ditampilkan berjalan dengan baik.



7. Kesimpulan

Berdasarkan percobaan dan analisis data dapat disimpulkan bahwa

- Canny Edge Detection adalah algoritma deteksi tepi yang dirancang untuk mengidentifikasi tepi dalam citra dengan akurasi tinggi.
- Algoritma Canny efektif dalam mendeteksi batas-batas fisik rel dengan mengolah citra menjadi peta tepi yang jelas, memudahkan ekstraksi fitur penting pada jalur kereta api.
- Metode stereo vision menghasilkan peta kedalaman (depth map) berdasarkan disparitas antara dua gambar stereo, memungkinkan pengukuran parameter tiga dimensi seperti lebar, panjang, kemiringan rel serta deformasi ballast secara real-time.

8. Saran

Memerhatikan keakuratan pengukuran serta kondisi lingkungan pengamatan (pencahayaan, letak pengambilan, dan sebagainya).

9. Daftar Pustaka

- Muttaqin, Arafah M., Jaya A.K., Suryawan M.A., Gustiana Z., Banjarnahor A.R., Bukidz D.P., Hazriani, Simanjuntak M., Saputra N., Fajrillah. 2023. *Implementasi Artificial Intelligence (AI) dalam Kehidupan*. Cetakan ke-1. Yayasan Kita Menulis. Langsa.
- Santoso J.T., 2023. *KECERDASAN BUATAN (Artificial Intelligence)*. Yayasan Prima Agus Teknik Bekerja sama dengan Universitas Sains & Teknologi Komputer (Universitas STEKOM). Semarang.
- Zhang, Y., Lu, Z., Zhang, X., Xue, J. H., & Liao, Q. (2021). Deep learning in lane marking detection: A survey. *IEEE Transactions on Intelligent Transportation Systems*, 23(7), 5976-5992.

LAMPIRAN

1. Program Setup Stereo Camera

```
import cv2

# Inisialisasi kamera kiri dan kanan

cam_left = cv2.VideoCapture(0)
cam_right = cv2.VideoCapture(1)

while True:

    retL, frameL = cam_left.read()
    retR, frameR = cam_right.read()

    if not retL or not retR:
        print("Error capturing video")
        break

    # Tampilkan hasil kamera stereo
    cv2.imshow("Left Camera", frameL)
    cv2.imshow("Right Camera", frameR)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cam_left.release()
cam_right.release()
cv2.destroyAllWindows()
```

2. Program Implementasi Canny Edge Detection

```
import cv2
import numpy as np

def canny_edge_detection(image):
    """Fungsi untuk mendeteksi tepi menggunakan Canny"""
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5, 5), 0)
    edges = cv2.Canny(blur, 50, 150)
    return edges

# Capture stereo images
cam_left = cv2.VideoCapture(0)
cam_right = cv2.VideoCapture(1)

while True:
    retL, frameL = cam_left.read()
    retR, frameR = cam_right.read()

    if not retL or not retR:
        break

    # Proses Canny Edge Detection
    edgesL = canny_edge_detection(frameL)
    edgesR = canny_edge_detection(frameR)

    # Tampilkan hasil
    cv2.imshow("Canny Left", edgesL)
```



```
cv2.imshow("Canny Right", edgesR)
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):  
    break
```

```
cam_left.release()
```

```
cam_right.release()
```

```
cv2.destroyAllWindows()
```

3. Program Estimasi Kedalaman & Geometri Rel

```
import cv2
```

```
import numpy as np
```

```
# Load stereo calibration parameters
```

```
stereo = cv2.StereoBM_create(numDisparities=16, blockSize=15)
```

```
def compute_depth(left_image, right_image):
```

```
    """Menghitung peta kedalaman dari dua gambar stereo"""
```

```
    grayL = cv2.cvtColor(left_image, cv2.COLOR_BGR2GRAY)
```

```
    grayR = cv2.cvtColor(right_image, cv2.COLOR_BGR2GRAY)
```

```
    disparity = stereo.compute(grayL, grayR)
```

```
    return disparity
```

```
# Capture video dari dua kamera
```

```
cam_left = cv2.VideoCapture(0)
```

```
cam_right = cv2.VideoCapture(1)
```

```
while True:
```

```
    retL, frameL = cam_left.read()
```

```
retR, frameR = cam_right.read()
```

```
if not retL or not retR:
```

```
    break
```

```
# Hitung kedalaman rel
```

```
depth_map = compute_depth(frameL, frameR)
```

```
# Tampilkan hasil
```

```
cv2.imshow("Depth Map", depth_map)
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

```
cam_left.release()
```

```
cam_right.release()
```

```
cv2.destroyAllWindows()
```